Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 2:
Server TCP Bloccante/Non-Bloccante

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Client: Generazione Dinamica 2
3.2 Server: Modelli a Confronto 3
3.3 Istruzioni di Esecuzione 3

4 Risultati **3**
4.1 Validazione Funzionale 4
4.2 Analisi Wireshark 4

5 Conclusioni e Sviluppi Futuri **4**

1 Problem Statement

L'obiettivo della seconda esercitazione (Hands-On 2) è l'analisi comparativa dei modelli di I/O nel contesto della programmazione di rete sotto Linux. Il *core* dell'attività consiste nell'implementazione di un servizio di validazione della **Successione di Fibonacci**. La sfida tecnica risiede nel gestire flussi di dati sequenziali in due scenari distinti: un server a modello **bloccante** (iterativo) e un server **non-bloccante** basato su multiplexing. Il sistema deve garantire che ogni numero ricevuto dal client segua rigorosamente la legge di ricorrenza $F_n = F_{n-1} + F_{n-2}$, restituendo un acknowledgment $(\texttt{ACK}_F IB_O K) osegnalidierroreincasodidivergenza.$

2 Stato dell'Arte

La gestione della concorrenza e delle attese è un pilastro dei sistemi distribuiti.

- **I/O Bloccante**: Rappresenta il comportamento di default dei socket Unix. Una chiamata a read() sospende il thread del chiamante finché il buffer di ricezione del kernel non contiene dati. Sebbene semplice, questo modello soffre di gravi problemi di scalabilità (scarsa *responsiveness*) in presenza di client lenti o multipli.

- **I/O Non-Bloccante**: Tramite la syscall fcntl($\texttt{O}_N ONBLOCK), ilsocketvieneistruitoanonsosp$
$Perevitareil$***busy-waiting***$(pollingcontinuo)susocketnon-bloccanti, siutilizzala$select(), $chepermet$

3 Metodologia ed Implementazione

L'implementazione prevede un client generatore e due varianti di server. La logica di validazione della sequenza di Fibonacci è il cuore algoritmico del sistema.

3.1 Client: Generazione Dinamica

Il client calcola i termini della successione e li trasmette serializzati come stringhe.

Logica Fibonacci Client

```
long long a = 0, b = 1, next;
for (int i = 0; i < MAX_FIB; i++) {
    sprintf(msg, "%lld", a);
    send(sock, msg, strlen(msg), 0);
    // ... attesa ACK ...
    next = a + b; a = b; b = next;
}
```

3.2 Server: Modelli a Confronto

Il server bloccante utilizza un semplice loop accept → read. Il server non-bloccante invece, configura i socket tramite:

Snippet Configurazione Non-Bloccante

```
int flags = fcntl(master_socket, F_GETFL, 0);
fcntl(master_socket, F_SETFL, flags | O_NONBLOCK);
activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);
```

Questa architettura permette di mantenere stati di validazione indipendenti per ogni client tramite una struttura client$_state$.

3.3 Istruzioni di Esecuzione

La compilazione dei modelli bloccanti e non-bloccanti avviene tramite GCC:

GCC Build

```
$ gcc server_tcp_block.c -o s_block
$ gcc server_tcp_nonblock.c -o s_nonblock
$ gcc client_tcp.c -o client
```

4 Risultati

La validazione è stata condotta simulando accessi concorrenti e ispezionando l'interlacciamento dei pacchetti.

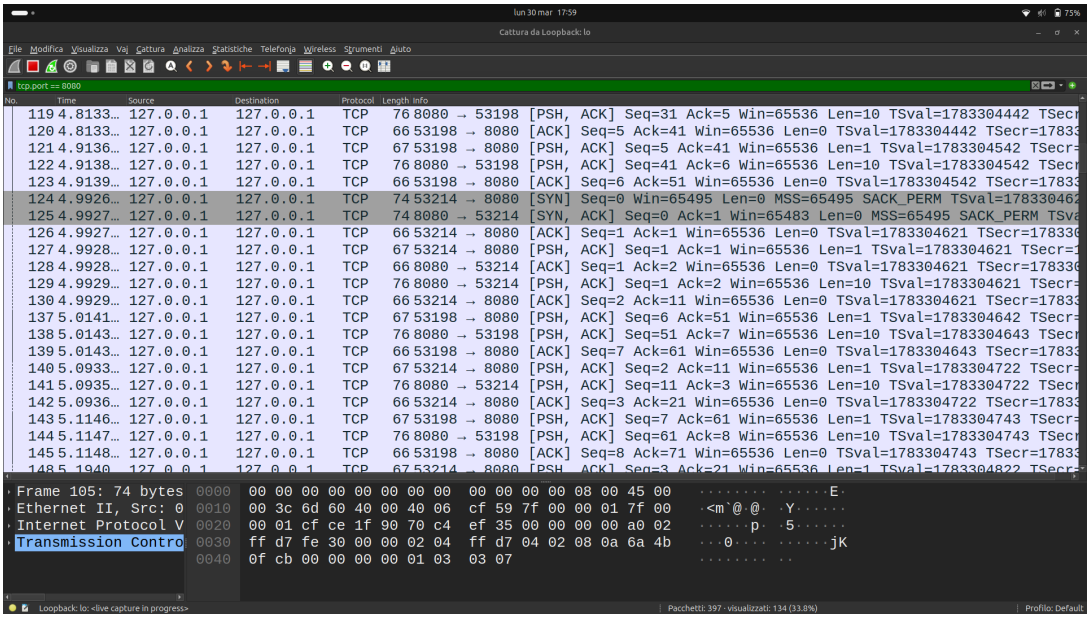

Figure 1: Monitoraggio socket non-bloccanti su Wireshark.

4.1 Validazione Funzionale

Entrambi i server riconoscono correttamente la sequenza di Fibonacci. Tuttavia, i log evidenziano la differenza nel determinismo temporale:

Log Multiplexing

```
New connection, socket fd: 4, ip: 127.0.0.1
Received: 0 from socket 4 -> ACK_FIB_OK
Received: 1 from socket 5 -> ACK_FIB_OK (Gestione Parallela)
```

4.2 Analisi Wireshark

L'analisi del traffico conferma che nel modello bloccante un secondo client rimane in attesa (nonostante l'handshake kernel sia avvenuto) finché il primo non chiude la sessione. Nel modello non-bloccante, i pacchetti PSH, ACK di diversi client risultano mescolati cronologicamente, dimostrando il successo del multiplexing.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 2 ha dimostrato che il multiplexing dell'I/O è essenziale per la creazione di server reattivi. La gestione manuale degli stati dei client (*state machines*) è un compromesso necessario per ottenere prestazioni elevate in un singolo thread.
Sviluppi futuri:

- **Passaggio a Epoll**: Evolvere verso epoll() per gestire migliaia di connessioni simultanee con complessità $O(1)$ invece di $O(N)$ della select().
- **Load Balancing**: Implementazione di una coda di messaggi per distribuire il carico di validazione computazionale su un pool di worker thread.